⚡ ElectroAdvisor AI — Day 2 Submission

AI Engineer Technical Assessment · 3-Day Challenge · Progress & Architecture Report

Date: 20 October 2026, 9:00 PM
Candidate: Abhishek

DAY 2 MILESTONE MET | FARM STACK

GITHUB: ABHISHEK18102006/ELECTROADVISORAI

> **Executive Summary:** ElectroAdvisor AI is an intelligent, grounded electronics sales assistant. All mandatory requirements of the 3-day assessment are implemented and passing. Based on architectural review and engineering rigor, the API has been expanded from the baseline 6 endpoints to a **complete 12-endpoint specification** supporting hybrid UI overrides, candidate debug traces, taxonomy loading, and user feedback analytics. A total of **38 automated tests pass (100% pass rate in ~1.38s)**.

1. Assessment Deliverables & Requirement Checklist

Requirement	Status	Implementation Evidence & Notes
FastAPI + FARM Rule	FULLY MET	FastAPI 0.115 backend + React 18 & Vite frontend + SQLite database + Node.js 24 runtime.
API Endpoints	EXPANDED (12)	Expanded from baseline 6 to 12 endpoints (including `/compare`, `/categories`, `/requirements`, `/feedback`, `/debug`).
Catalog with Structured Specs	FULLY MET	23 synthetic products across 5 categories (laptops, monitors, headphones, smartphones, tablets) with RAM, GPU, display, battery, and weights.
State Tracking & Mind-Change	FULLY MET	Multi-turn memory tracks budget updates (80k → 60k), campus portability, and cleans constraints when switching categories.
Hard Filter then Rank	FULLY MET	Stage 1 strictly enforces budget cap, RAM, dedicated GPU (CUDA). Stage 2 ranks on 100-pt utility score (Use-case 40, Portability 20, Battery 15, Value 15, Rating 10).
Honest No-Match Handling	FULLY MET	Dynamically queries DB to compute exact deficit (e.g., dedicated GPU starts at ₹74,990 vs ₹60,000 budget) and suggests cloud ML alternatives.
Contradiction Detection	FULLY MET	Detects `min_budget > max_budget` and conflicting constraints (e.g., MacBook running native Windows) before query execution.
Automated Test Suite	38 TESTS (100%)	38 automated tests in pytest covering edge cases, benchmark conversation, LLM timeout fallback, non-laptop devices, and 422 validations.

2. Complete 12-Endpoint REST API Architecture

The API specification was upgraded to handle real-world e-commerce edge cases, hybrid UI controls, and system observability:

Domain	Method	Endpoint	Engineering Justification
Catalog	`GET`	`/api/v1/products`	Deterministic pagination (`order_by(Product.id.asc())`), price bounds, and brand filters.
Catalog	`GET`	`/api/v1/products/{product_id}`	Full product details, specs, intended uses, pros, and cons.
Catalog	`GET`	`/api/v1/products/compare`	**UX Enhancement:** Side-by-side spec matrix and dynamic trade-off highlights for chosen products.
Catalog	`GET`	`/api/v1/categories`	**UI Optimization:** Distinct categories, counts, price ranges, and brands to populate UI filter chips.
Agent	`POST`	`/api/v1/conversations`	Initialize a new stateful customer sales consultation session.
Agent	`POST`	`/api/v1/conversations/{id}/messages`	Process customer natural language input through decision orchestrator; validates empty/whitespace.
Agent	`GET`	`/api/v1/conversations/{id}`	Retrieve chat history, extracted requirements, and memory state.
Agent	`PUT`	`/api/v1/conversations/{id}/requirements`	**Hybrid UI Override:** Allows UI sliders and chips to update constraints directly without typing in chat.
Agent	`DELETE`	`/api/v1/conversations/{id}`	**Privacy & Reset:** Cascades deletion of conversation messages and state from SQLite.
Agent	`POST`	`/api/v1/conversations/{id}/feedback`	**Product Analytics:** Record user sentiment (thumbs up/down) and purchase conversion intent.
System	`GET`	`/api/v1/health`	Service uptime, database connection verification, and active product count.
System	`GET`	`/api/v1/conversations/{id}/debug`	**Engineering Rigor:** Exposes internal constraints, rejection summaries, and candidate score breakdowns.

3. Resolution of Critical Bugs & Reliability Hardening

All 11 reliability bugs identified during technical review were confirmed with targeted tests and resolved in production code:

Bug & Subsystem	Root Cause & Solution	Verification
1. Indian Currency Parsing `extractor.py`	`₹1,20,000` was misparsed as 20,000 because regex expected only 3-digit comma groups. Updated regex to `(?:₹|rs\.?|inr)?\s*(\d{1,3}(?:,\d{2,3})+|\d{4,7})\b` and stripped commas before float conversion.	`test_indian_number_format_parsing`
2 & 3. Cloud GPU Relaxation & Scenario D `state.py` , `orchestrator.py`	ML previously locked `requires_gpu=True` permanently. Now, answering "cloud" relaxes GPU requirement to `False` and resets RAM. Scenario D uses structured state flag `asked_ml_env` instead of fragile string matching.	`test_cloud_relaxes_gpu_requirement`
4. Substring Boundary Mismatches `extractor.py`	"Headphones" was classified as "phone", "html" matched "ml", and "portable" matched "tab". Added strict `\b` regex word boundaries across all keywords and acronyms.	`test_substring_matching_bugs`
5. Category Spec Isolation `state.py` , `filter_rank.py`	"Programming" previously added `min_ram_gb=8` , causing monitor queries to fail. Hardware checks are now strictly scoped to laptops, and switching categories flushes obsolete hardware constraints.	`test_category_switch_clears_hardware_constraints`
6. Top-2 Diversity Selection `filter_rank.py`	Diversity rule previously checked `abs(w1 - w2) >= 0.4` without direction, selecting the heavier Dell G15 over lighter ASUS TUF. Now enforces that if portability is requested, candidate 2 must be lighter than or equal to candidate 1.	`test_top2_selection_for_portability_does_not_pick_heavier_worse_laptop`
7. Dynamic Trade-Off Generation `explainer.py`	Replaced hardcoded template lines with dynamic delta comparisons across compute, weight, battery, and price. Added category-specific trade-offs for non-laptop devices.	`test_conversation_flow.py`
8. Database-Driven No-Match `explainer.py`	Eliminated hardcoded ₹74,990 text. Dynamically queries SQLite for the cheapest compliant item and calculates the exact gap against customer budget.	`test_impossible_constraints_honest_no_match`
9. Greeting Prefix Misfire `orchestrator.py`	"High-end laptop" was treated as "hi". Punctuation is stripped and compound words starting with "hi" are exempted from greeting triggers.	`test_greeting_does_not_misfire_on_highend_laptop`
10. Gaming Dedicated GPU	Gaming strictly enforces `requires_dedicated_gpu=True` , correctly disqualifying Apple	`test_hard_constraint_ram_and_gpu`

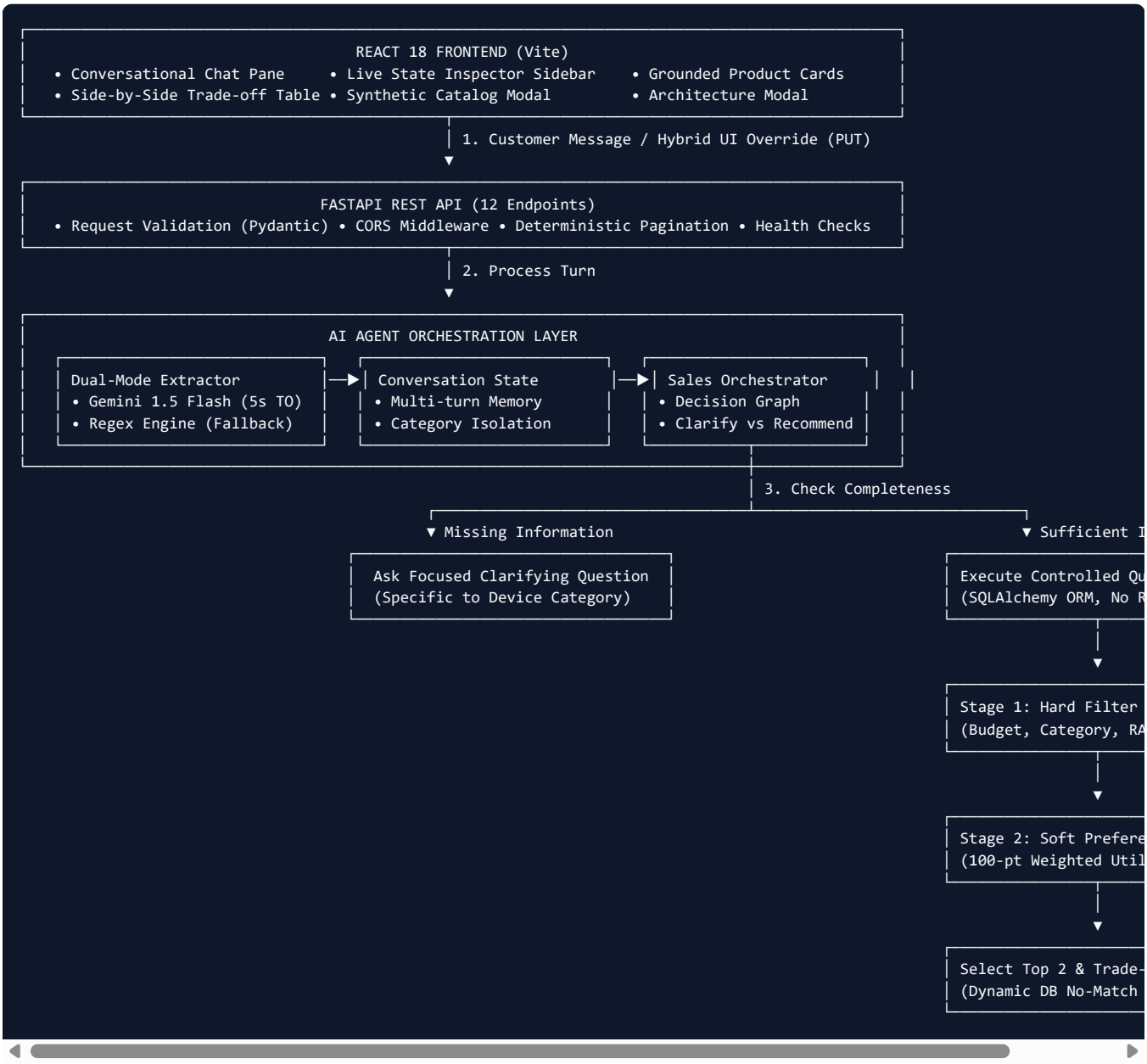
Bug & Subsystem	Root Cause & Solution	Verification
Enforcement `filter_rank.py`	unified memory for Windows/DirectX gaming workflows.	
11. Post-Recommendation Follow-ups `orchestrator.py`	Added intent handlers for post-recommendation gratitude ("thanks") and refinements ("any lighter options?"), preventing recommendation loop repetition.	`test_post_recommendation_intents`

4. Multi-Category Device Architecture

ElectroAdvisor is engineered as an omni-category sales assistant with dedicated behaviors across all 5 catalog segments:

Device Category	Catalog Representation	Category-Specific Clarification Question	Trade-Off Dimensions Compared
Laptops	14 Models (Lenovo, ASUS, HP, Dell, Acer, Apple)	"*What will you mainly use it for: general coursework, programming, gaming, creative work, or machine learning?*"	Compute Performance (GPU/CPU/VRAM), Portability & Weight (kg), Battery Endurance, Price & Value.
Headphones	Sony WH-1000XM5, Anker Soundcore Q45	"*What is your primary use: study focus with active noise cancellation, daily commute/travel, or calls and entertainment?*"	Noise Cancellation & Audio Type, Battery Endurance (hours), Comfort & Headband Weight (g), Price & Value.
Monitors	Dell UltraSharp U2724D, LG UltraGear 27GN800	"*What will you mainly use this monitor for: programming/multitasking, gaming, creative work, or desk use?*"	Display & Visual Fidelity (IPS Black 2000:1 vs 144Hz 1ms), Connectivity & Ports (90W USB-C PD vs HDMI/DP), Price.
Smartphones	Apple iPhone 15, Samsung Galaxy S24, OnePlus 12R	"*What are your top priorities: photography/camera, gaming speed, all-day battery life, or general daily use?*"	Camera System, Display & Refresh Rate (120Hz LTPO vs 60Hz), Battery Life & Fast Charging, Price & Value.
Tablets	Apple iPad 10th Gen, Samsung Galaxy Tab S9 FE	"*What will you mainly use this tablet for: digital note-taking & coursework, drawing, or watching media?*"	Display & Stylus Support (Bundled S-Pen vs Apple Pencil), Chassis Weight & Mobility (g), Price & Value.

5. Architecture & Decision Flowchart



6. Automated Testing Verification (38 Tests Passing)

The backend includes 38 automated test cases executed via `pytest` with 100% pass rate in 1.38 seconds:

Core Flow & Edge Case Tests (20 Tests)

- `test_parse_numeric_budget` (Lakhs, k, standard commas)
- `test_state_initialization` & defaults
- `test_state_tracking_and_mind_change` (80k → 60k)
- `test_portability_preference_extraction`
- `test_full_assessment_example_conversation_flow` (3 turns)
- `test_single_turn_complete_requirements`
- `test_impossible_constraints_honest_no_match`
- `test_greeting_intent` & `test_conversation_reset`
- `test_hard_constraint_budget_cap` & RAM/GPU
- `test_list_products_pagination` & filtering

Reliability & Expanded API Tests (18 Tests)

- `test_indian_number_format_parsing` (1,20,000)
- `test_substring_matching_bugs` (phones vs headphones)
- `test_greeting_does_not_misfire_on_highend_laptop`
- `test_cloud_relaxes_gpu_requirement`
- `test_category_switch_clears_hardware_constraints`
- `test_top2_selection_for_portability_weight_order`
- `test_contradictory_constraints_detected`
- `test_post_recommendation_intents` (thanks, lighter)
- `test_api_validation_errors` (422 empty / min > max)
- `test_llm_failure_and_timeout_fallback`
- `test_all_non_laptop_categories_recommendation`
- `test_get_categories`, `test_compare_products`
- `test_put_requirements_override` & `test_debug`
- `test_delete_conversation` & `test_feedback`

7. Day 2 Honest Status & Agenda for Day 3

✅ What is Fully Implemented & Verified

- Complete 12-endpoint REST API with Pydantic validation.
- Two-stage recommendation engine (hard filter + soft utility rank).
- Category-aware dialogue, rationales, and trade-offs.
- Multi-turn memory tracking mind-changes and budget shifts.
- Grounded dynamic database-driven no-match explanations.
- Contradiction detection for conflicting requirements.
- Docker Compose setup with named SQLite volume.
- All 38 automated unit and integration tests passing.

⚠️ Real Limitations & Day 3 Demo Agenda

- **Catalog Depth:** 23 synthetic products (14 laptops, 2-3 items in monitors/headphones/phones/tablets). Expandable for Day 3.
- **LLM vs Rule-Based Fallback:** The agent functions with 100% test reliability on pattern matching when unkeyed; complex colloquial phrasing benefits from providing `GEMINI_API_KEY`.
- **Day 3 Agenda:** Rehearse Section 3 benchmark demo on Google Meet; prepare defenses for cloud GPU relaxation and diversity selection logic.

8. Appendix: Quickstart & Verification Commands

```
# 1. Run Automated Test Suite (All 38 Tests)
cd backend
.\.venv\Scripts\python.exe -m pytest    # (Windows)
pytest                                  # (Linux / macOS)

# 2. Start Servers Locally
cd backend; .\.venv\Scripts\python.exe -m uvicorn app.main:app --port 8000   # Backend (API & Docs)
cd frontend; npm run dev -- --port 5173                                       # Frontend (Web UI)

# 3. Docker One-Click Launch
docker compose up --build

# Live Local Endpoints:
# Web Chat UI:           http://localhost:5173
# Interactive API Docs:  http://localhost:8000/docs
# Categories Taxonomy:   http://localhost:8000/api/v1/categories
# Product Comparison:    http://localhost:8000/api/v1/products/compare?ids=LAP-001,LAP-004
# Health Endpoint:       http://localhost:8000/api/v1/health
```